

PHASE 2 — SINGLE DOOR-NODE PROTOTYPE

Breadboard Wiring Guide & Bench Test Checklist

1. Scope

This document covers assembling and bench-testing one door node — the ESP32, credential readers, lock actuator, and sensors from the System Design Document — on a breadboard. The goal is to validate the hardware and firmware loop (credential read → decision → unlock → log) on a single node before committing to a PCB or scaling to multiple doors.

The accompanying firmware project (door_node_firmware/) implements the pin map and state machine referenced throughout this guide.

2. Bill of Materials — This Prototype

Item	Qty	Notes
ESP32 dev board (ESP32-WROOM-32)	1	Any common dev board breakout works
PN532 NFC/RFID module	1	SPI mode (check module's mode-select jumper/switch)
R307 or AS608 fingerprint module	1	UART interface
Magnetic reed switch	1	Door position sensor
Momentary push button	1	Stands in for the REX button
Momentary push button or micro switch	1	Stands in for the tamper switch
5V relay module (opto-isolated)	1	Driven by ESP32 GPIO; do not drive the strike directly from a GPIO pin
12V electric strike, OR a 12V LED as a bench stand-in	1	Use an LED first bench pass before wiring a real strike
MAX485 TTL-to-RS485 module	1	Not exercised by firmware logic yet; wired so the hardware path is validated now
10kΩ resistors	2	External pull-ups for the reed switch and REX button (input-only pins have no internal pull)
Breadboard + jumper wires	-	
12V bench power supply (separate from ESP32's 5V/USB supply)	1	Powers the relay coil and strike/LED, not the ESP32 logic

3. Pin Assignment

Matches include/config.h in the firmware project exactly. If you change a pin here, change it there too.

Signal	ESP32 Pin	Notes
PN532 SCK	GPIO18	VSPI bus
PN532 MISO	GPIO19	VSPI bus
PN532 MOSI	GPIO23	VSPI bus
PN532 SS (CS)	GPIO5	Strapping pin, but standard VSPI CS0 default — fine in practice
Fingerprint RX	GPIO16	ESP32 UART2
Fingerprint TX	GPIO17	ESP32 UART2
MAX485 RX (module RO)	GPIO4	Remapped UART1
MAX485 TX (module DI)	GPIO32	Remapped UART1
MAX485 DE/RE	GPIO33	Tie DE and RE pins together on the module
Reed switch (door position)	GPIO34	Input-only pin — needs the external 10k pull-up
REX button	GPIO35	Input-only pin — needs the external 10k pull-up
Tamper switch	GPIO27	Uses internal pull-up (set in firmware)
Lock relay control	GPIO26	Drives the relay module's IN pin, not the strike directly
Status LED	GPIO13	Optional onboard indicator

4. Assembly Steps

1. Power the ESP32 from USB only for steps 2–6. Leave the 12V bench supply switched off until Section 6.
2. Wire the PN532 to the VSPI pins (18/19/23/5) per the table above. Confirm the module is set to SPI mode, not I2C — most PN532 breakouts have a jumper or switch for this.
3. Wire the fingerprint module's RX/TX to ESP32 GPIO17/GPIO16 (cross them: module RX to ESP32 TX pin, module TX to ESP32 RX pin).
4. Wire the reed switch between GPIO34 and GND, with a 10kΩ pull-up resistor from GPIO34 to 3.3V.
5. Wire the REX push button the same way, between GPIO35 and GND with a 10kΩ pull-up to 3.3V.
6. Wire the tamper micro switch between GPIO27 and GND (no external resistor needed — firmware enables the internal pull-up).
7. Wire the MAX485 module's RO/DI/DE/RE to GPIO4/GPIO32/GPIO33 (DE and RE tied together). Leave the A/B bus lines unconnected for now — there is no second node to talk to yet.
8. Wire the relay module: IN to GPIO26, VCC to 5V, GND to ESP32 GND. Do not connect the relay's high-voltage side yet.
9. Flash the firmware (pio run --target upload) with secrets.h filled in and confirm serial output shows PN532 and fingerprint sensors initializing OK before continuing.

5. Before First Power-Up — Safety Checklist

- The 12V supply and the ESP32's USB/5V supply share a common ground, but are otherwise separate circuits. Confirm this before connecting anything to the relay's switched side.
- If testing with a real 12V electric strike instead of an LED stand-in: confirm the relay module is rated for the strike's inrush current, and that a flyback diode is present across the strike coil (most electric strikes need one; check the datasheet).

- Do not touch the relay's high-voltage terminals while powered.
- Start every test with the LED stand-in, not the real strike. Move to the real strike only after Section 6's timing test passes with the LED.

6. Bench Test Checklist (P2.8 – P2.10)

6.1 Lock actuation timing (P2.8)

- Present an allow-listed card UID (from the allowList array in main.cpp, or add your own test card's UID).
- Expected: serial log shows [CARD] UID read -> [ACCESS] Granted, unlocking, and the relay/LED energizes within about 200 ms of the card read.
- Measure and record how long the relay stays energized — should match UNLOCK_HOLD_MS (5000 ms) within firmware timing tolerance.
- Present a card UID that is not in the allow-list; confirm serial log shows [ACCESS] Denied and the relay does not energize.

6.2 Power & isolation validation (P2.9)

- With the relay coil side powered from the ESP32's 5V rail and the switched side from the separate 12V supply, confirm the ESP32 does not reset or brown out when the relay switches (a poorly isolated relay can inject noise back into the logic supply).
- Measure voltage on the 3.3V rail during relay switching with a multimeter or scope; it should stay within the ESP32's tolerance (no visible dip below ~3.0V).
- Confirm the reed switch and REX button read the correct logic level in both door/button states — verify polarity against what MONITOR_DOOR in main.cpp expects, and adjust the doorOpen comparison if your wiring is inverted.

6.3 Single-node integration test (P2.10)

- Full loop: present a valid card -> confirm unlock -> toggle the reed switch to simulate the door opening and closing -> confirm relock after UNLOCK_HOLD_MS -> confirm an access_event MQTT message was published for the attempt (check with a tool such as MQTT Explorer or mosquitto_sub against your test broker).
- Tamper test: trigger the tamper switch mid-cycle (during IDLE or during MONITOR_DOOR) and confirm a tamper alert publishes immediately, regardless of the current state.
- REX test: press the REX button with no card presented; confirm it unlocks immediately without going through CARD_SCAN.
- Network-loss behavior: disconnect Wi-Fi mid-test and confirm the state machine keeps running locally (card reads still unlock the door) even though MQTT publishing fails — this is the expected degraded-but-functional behavior for a single node before the RS-485 fallback exists.
- Record results (pass/fail per item above) before moving to Phase 3 (backend) and Phase 6 (multi-node scaling).

7. Known Limitations at This Phase

- Card authentication is UID-only; DESFire secure authentication is a Phase 5 deliverable.
- The credential allow-list is hardcoded in firmware; Phase 3 replaces it with a real backend lookup.
- RS-485 hardware is wired and initialized but not yet exercised by firmware logic; that begins in Phase 6.
- This firmware was reviewed line-by-line against the Adafruit PN532, Adafruit Fingerprint, PubSubClient, and ArduinoJson v7 APIs, but has not been compiled against the real ESP32 toolchain in this environment (the Espressif platform download did not complete in the sandbox). Run pio run as your first bench step and treat any compiler errors as the actual source of truth over this document.